

实验三：语义分析实验

一、实验目的

结合语法分析过程，分析输入程序的语义，加深对语义分析的理解。

二、实验内容

- (1) 描述类型推断和类型检查过程；
- (2) 编制语义分析程序；如果输入的源代码没有类型方面的语义错误，输出PASS，否则输出ERROR。

类型

(1) 数据类型：有 32 位**整型**（数值）与**数组**（本质为指针）

只支持一维数组，不判断越界

(2) 类型推断：变量类型由其在源文件中首次出现的形式决定（例如 `a = 0` 推断为整数，`a[] = 10` 推断为数组）。

a) 数组声明：`a[] = 10;` // 表示申请 10 个单位（每个 4 字节）的堆空间。

b) 字符串赋值：`s[] = "Hello";` // 自动根据字符串长度申请空间并初始化。

(3) 作用域：引入 `local` 与 `global` 关键字，支持基于 `{}` 的局部变量管理

a) 全局变量：在函数外定义，或使用 `global` 显式声明。**未经声明且直接使用的变量默认为全局。**

b) 局部变量：在函数内通过 `local` 声明。可见域从声明处开始，到所属的代码块 `{}` 结束。

类型推断

全局变量或局部变量在首次声明或使用时根据上下文推断类型，具体规则如下：

- 有以下情况之一的，推断为整数变量：
 - 作为右值的，如 `a = b + 1;` 中的 `b`，推断为全局整数变量并赋予初始值 0。
 - `a = expr;`、`local a;` 或 `global a;` 且 `expr` 是结果为整数的表达式（字面量、变量、函数调用等）。
 - `a = read;` 且 `a` 未定义，则等价于先执行 `global a;` 再执行 `a=read;`，类型推断为整数变量。
- 有以下情况之一的，推断为数组变量：
 - `a = expr;`、`local a[];` 或 `global a[];` 且 `expr` 是结果为数组的表达式（函数调用、字符串字面值）。
 - `a[] = expr;` 且 `expr` 是结果为整数的表达式（字面值、变量、函数调用等）。
 - 如 `a[] = read;` 且 `a` 未定义，则等价于先执行 `global a[];` 再执行 `a[] = read;`，类型推断为数组变量¹。

¹特别地，`a="Hello";` 与 `a[]="Hello";` 均推断为数组变量 `a`，并根据字符串长度动态分配内存存储 UTF-32 编码的 C 风格字符串内容。

输入和输出

(1) write:

a) `write = 整数`; 以十进制格式输出数值。

b) `write = 数组`; 将数组内容作为 UTF-32 字符串输出（考虑到多数终端与文件编码为 UTF-8, 故这里需要进行一个编码转换）。

(2) read（始终触发动态内存申请）：

a) 首次出现: `a[] = read`; 声明a为数组并读入字符串。

b) 覆盖赋值: `a = read`; 如果a是数组则释放a原有的空间，申请新空间存入新字符串。

否则从stdin读入一个整数。

(3) `get / put`: 读取或输出单个字符，读入需从UTF-8转换为UTF-32，输出则需要把UTF-32转换为UTF-8。

测试用例1分析(ERROR)

`a[] = "test";` `// a="Hello"; 与 a[]="Hello"; 均推断为数组变量 a`

`a = 5;` `// 对数组变量赋整型值`

测试用例2分析(ERROR)

```
a[] = "test";
```

```
a = get;
```

```
// 对数组型变量直接执行 get、put 操作
```

```
put = a;
```

```
// 对数组型变量直接执行 get、put 操作
```

测试用例3分析(ERROR)

```
func[] foo(a[]) {  
    return a;  
}
```

```
b = 10;
```

```
ignore = foo(b);
```

// 把整型变量赋给数组型参数

测试用例4分析(ERROR)

```
func sum(a, b) {  
    return a + b;  
}
```

```
x[] = "hello";
```

```
write = sum(x, 50);    // 把整型变量赋给数组型参数
```

```
put = '\n';
```


测试用例5分析(PASS)

write = read + read;

put = '\n';

文法 (2)

```
Statement      ::= AssignmentStmt
                  | IfStmt
                  | WhileStmt
                  | JumpStmt
                  | DeclarationStmt
                  | Block
                  | ";"

Block           ::= "{" { Statement } "}"

IfStmt          ::= "if" "(" Expression ")" Block "else" Block

WhileStmt       ::= "while" "(" Expression ")" Block

AssignmentStmt  ::= LValue "=" Expression ";"
                  | Identifier "[" "]" "=" Expression ";" /* 数组声明或初始化 */

JumpStmt        ::= "break" ";"
                  | "continue" ";"
                  | "return" Expression ";"

DeclarationStmt ::= ( "local" | "global" ) Identifier [ "[" "]" ] ";"
```

文法 (3)

```
LValue      ::= Identifier [ "[" Expression "]" ]  
              | "write"  
              | "put"  
  
Expression  ::= LogicalExp  
  
LogicalExp  ::= RelationalExp { ("==" | "!=") RelationalExp }  
  
RelationalExp ::= AdditiveExp { ("<" | "<=" | ">" | ">=") AdditiveExp }  
  
AdditiveExp  ::= MultiplicativeExp { ("+" | "-") MultiplicativeExp }  
  
MultiplicativeExp ::= Primary { ("*" | "/" | "%") Primary }  
  
Primary      ::= Integer  
              | CharConstant  
              | StringConstant  
              | Identifier "(" [ ArgumentList ] ")" /* 函数调用 */  
              | LValue  
              | "read"  
              | "get"  
              | "(" Expression ")"  
  
ArgumentList ::= Expression { "," Expression }
```